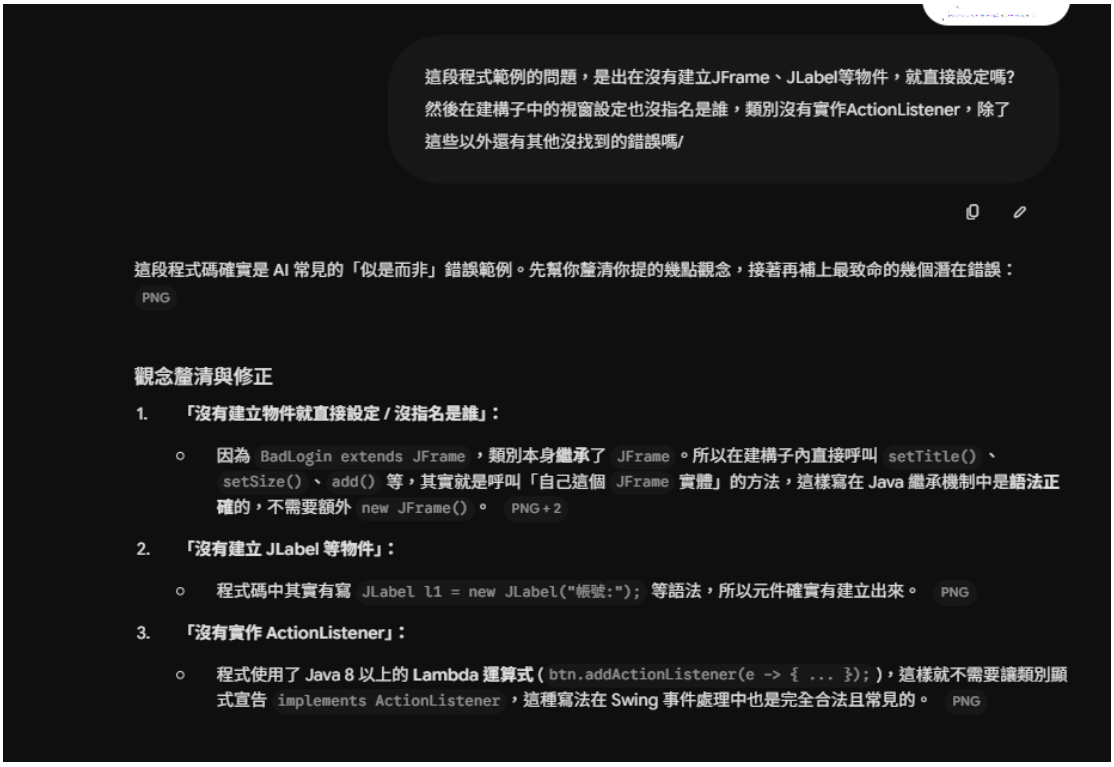
學號:A1143345 姓名:蔡承佑 作業:登入與登出視窗改錯

( AI 對話紀錄 )






### 3. 邏輯與體驗瑕疵：`setVisible(true)` 呼叫時機過早

Java

```
setVisible(true); // 寫在建構子最前面
setLayout(null);
// ... 建立與 add 元件 ...
```

- **問題**：在元件還沒建立完成、還沒 `add()` 到視窗前就呼叫 `setVisible(true)` [cite: 1]。
- **結果**：容易造成畫面繪製異常（Render Issue），有時元件新增後不會自動刷出來，需要手動縮放視窗才會出現。`setVisible(true)` 應放在所有元件都 `add` 完畢的最後一行。

### 4. 缺少程式關閉機制（Default Close Operation）

- **問題**：沒有設定 `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);`。
- **結果**：使用者按下視窗右上角的「X」關閉視窗時，視窗雖然隱藏了，但 Java 程式進程（Process）依然在背景繼續執行、不會釋放資源。

### 5. 安全性與UI元件選擇問題（密碼欄位）

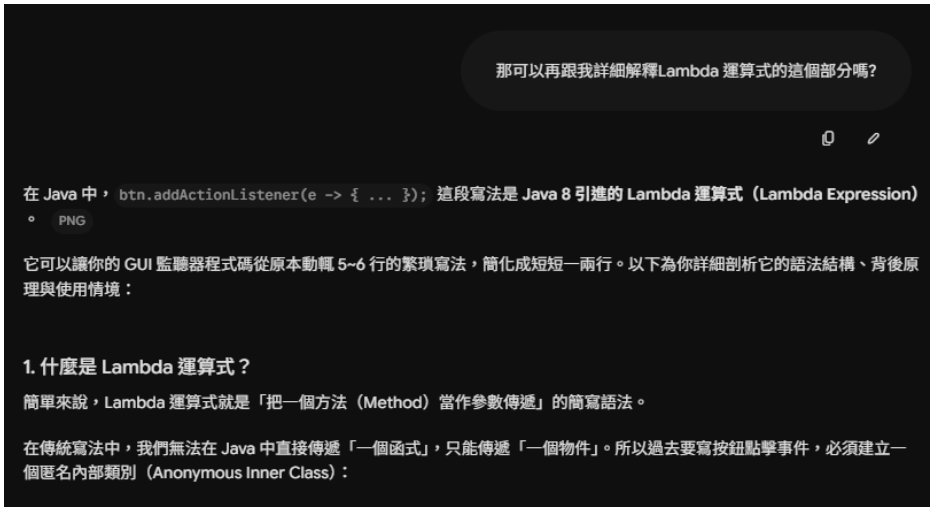
Java

```
JTextField t2 = new JTextField();[cite: 1]
```

- **問題**：密碼欄位使用了明文顯示的 `JTextField` [cite: 1]。
- **修正**：密碼輸入框應使用 `JPasswordField`，才能將輸入內容隱蔽（顯示為圓點或星號）。

(反思與後續學習)

與 exercise1 的作業相同，我都是先提出自己的答案或疑問，再跟 AI 確認是否正確，且因為我在最初提到的幾個點都是誤會，所以後來又針對 Lambda 運算式的部分，進行了後續的追問



傳統寫法（匿名內部類別）：

Java

```
btn.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        if ("admin".equals(t1.getText()) && "".equals(t2.getText())) {
            System.out.println("登入成功");
        }
    }
});
```

Lambda 寫法（圖片中的寫法）：

Java

```
btn.addActionListener(e -> {
    if (t1.getText() == "admin" && t2.getText() == "") // (先不論字串比較的邏輯錯誤)
        System.out.println("登入成功");[cite: 1]
});
```

你看，`new ActionListener()`、`@Override`、`public void actionPerformed` 這些繁瑣的模板程式碼全部都不見了！

## 2. Lambda 語法結構解析

Lambda 的核心結構由三部分組成：

(參數) → {執行程式碼}

以 `e -> { ... }` 為例：

1. **參數** `e`：
   - 代表傳入的 `ActionEvent` 物件（裡面包含按鈕被點擊時的詳細資訊，例如觸發源、時間等）。
   - **型別自動推導**：我們不需要寫成 `(ActionEvent e)`，因為 Java 編譯器知道 `addActionListener` 接收的是 `ActionListener`，而它的方法參數就是 `ActionEvent`，所以寫 `e` 就好。
   - 當參數**只有一個**時，括號 `()` 可以省略。
2. **箭頭** `->`：
   - Lambda 的專屬符號，意思是「把左邊的參數，傳給右邊的主體執行」。
3. **主體** `{ ... }`：
   - 被觸發時要執行的程式碼區塊。如果區塊內只有單一行敘述，甚至連大括號 `{}` 也可以省略。

## 3. 為什麼不需要 `implements ActionListener`？

你原本提到「類別沒有實作 ActionListener」，這是因為 `ActionListener` 在 Java 中是一個**功能性介面（Functional Interface）**。

- **功能性介面**：指的是「內部只有唯一一個抽象方法」的介面（`ActionListener` 裡面只有一個 `actionPerformed(ActionEvent e)`）。
- **背後運作機制**：
  編譯器看到 `btn.addActionListener(...)` 時，知道這裡需要一個 `ActionListener` 物件。當你寫入 `e -> { ... }` 時，編譯器會自動把這個 Lambda 綁定到 `actionPerformed` 這個唯一的方法上。

因此，**類別本身完全不需要宣告** `implements ActionListener`，也不需要自己寫 `actionPerformed` 方法。




然後也是在這個作業中，發覺了程式的架構似乎不同於課本上的範例，他將職責分開，因此我在 execerise1 的作業中，也才嘗試了這種架構的寫法